

MIP_SQLITE_SCHEMA.md

Mainframe Intelligence Platform

Physical Persistence Architecture

Version: 1.0

Status: Foundational Platform Specification

Classification: Core Platform

Purpose

This document defines the physical database schema used by MIP.

The schema stores:

- Discovery Results
- Metadata
- Relationships
- Evidence
- Validation Results
- Reasoning Outputs
- Modernization Intelligence

The schema is designed for:

SQLite

↓

PostgreSQL

↓

Enterprise Scale Deployment

without requiring redesign.

Architecture Principles

Metadata First

Metadata is the source of truth.

Graph Friendly

Relationships stored explicitly.

Evidence Driven

Every fact must have evidence.

AI Ready

Schema must support:

- Agent Queries
 - RAG
 - Reasoning
 - Recommendations
-

Database Overview

```
mip.db
```

```
|— repository
|— application
|— domain
|— capability
|— program
|— job
|— job_step
|— copybook
|— dataset
|— table_metadata
|— column_metadata
|— transaction_metadata
|— relationship
|— evidence
```

```
|— validation_result
|— modernization_recommendation
```

Repository

Represents a scanned repository.

```
CREATE TABLE repository (
  repository_id TEXT PRIMARY KEY,
  repository_name TEXT NOT NULL,
  repository_path TEXT NOT NULL,
  scan_date TIMESTAMP,
  created_at TIMESTAMP
);
```

Application

```
CREATE TABLE application (
  application_id TEXT PRIMARY KEY,
  application_name TEXT NOT NULL,
  description TEXT,
  owner TEXT,
  criticality TEXT,
  created_at TIMESTAMP
);
```

Domain

```
CREATE TABLE domain (
  domain_id TEXT PRIMARY KEY,
  application_id TEXT NOT NULL,
  domain_name TEXT NOT NULL,
  description TEXT,

  FOREIGN KEY(application_id)
    REFERENCES application(application_id)
);
```

Capability

```
CREATE TABLE capability (  
  capability_id TEXT PRIMARY KEY,  
  domain_id TEXT NOT NULL,  
  capability_name TEXT NOT NULL,  
  description TEXT,  
  
  FOREIGN KEY(domain_id)  
    REFERENCES domain(domain_id)  
);
```

Program

Most important table.

```
CREATE TABLE program (  
  
  program_id TEXT PRIMARY KEY,  
  
  program_name TEXT NOT NULL,  
  
  program_type TEXT,  
  
  language TEXT,  
  
  source_path TEXT,  
  
  lines_of_code INTEGER,  
  
  complexity_score INTEGER,  
  
  called_program_count INTEGER,  
  
  copybook_count INTEGER,  
  
  table_count INTEGER,  
  
  dataset_count INTEGER,  
  
  created_at TIMESTAMP  
);
```

Indexes

```
CREATE INDEX idx_program_name  
ON program(program_name);
```

```
CREATE INDEX idx_program_type  
ON program(program_type);
```

Job

```
CREATE TABLE job (  
    job_id TEXT PRIMARY KEY,  
    job_name TEXT NOT NULL,  
    description TEXT,  
    step_count INTEGER,  
    schedule_type TEXT,  
    created_at TIMESTAMP  
);
```

Job Step

```
CREATE TABLE job_step (  
    step_id TEXT PRIMARY KEY,  
    job_id TEXT NOT NULL,  
    step_name TEXT,  
    sequence_number INTEGER,  
    executed_program TEXT,  
    FOREIGN KEY(job_id)  
        REFERENCES job(job_id)  
);
```

Copybook

```
CREATE TABLE copybook (  
    copybook_id TEXT PRIMARY KEY,  
    copybook_name TEXT NOT NULL,  
    source_path TEXT,  
    field_count INTEGER,  
    record_count INTEGER  
);
```

Dataset

```
CREATE TABLE dataset (  
    dataset_id TEXT PRIMARY KEY,  
    dataset_name TEXT NOT NULL,  
    dataset_type TEXT,  
    owner TEXT,  
    retention_policy TEXT  
);
```

Table Metadata

(DB2)

```
CREATE TABLE table_metadata (  
    table_id TEXT PRIMARY KEY,  
    schema_name TEXT,  
    table_name TEXT,
```

```
criticality TEXT,  
  
row_count INTEGER  
);
```

Column Metadata

```
CREATE TABLE column_metadata (  
  
    column_id TEXT PRIMARY KEY,  
  
    table_id TEXT NOT NULL,  
  
    column_name TEXT,  
  
    data_type TEXT,  
  
    nullable INTEGER,  
  
    FOREIGN KEY(table_id)  
        REFERENCES table_metadata(table_id)  
);
```

Transaction Metadata

```
CREATE TABLE transaction_metadata (  
  
    transaction_id TEXT PRIMARY KEY,  
  
    transaction_name TEXT,  
  
    transaction_type TEXT,  
  
    entry_program TEXT  
);
```

Relationship

This table powers graph generation.

```
CREATE TABLE relationship (  
    relationship_id TEXT PRIMARY KEY,  
    source_type TEXT NOT NULL,  
    source_id TEXT NOT NULL,  
    relationship_type TEXT NOT NULL,  
    target_type TEXT NOT NULL,  
    target_id TEXT NOT NULL,  
    confidence_score REAL,  
    created_at TIMESTAMP  
);
```

Relationship Types

```
CALLS  
EXECUTES  
USES_COPYBOOK  
READS_TABLE  
WRITES_TABLE  
READS_DATASET  
WRITES_DATASET  
PRODUCES  
CONSUMES  
STARTS_TRANSACTION  
BELONGS_TO_CAPABILITY  
BELONGS_TO_DOMAIN
```


Evidence

Every extracted fact must be explainable.

```
CREATE TABLE evidence (  
    evidence_id TEXT PRIMARY KEY,  
    entity_type TEXT,  
    entity_id TEXT,  
    source_file TEXT,  
    source_type TEXT,  
    line_start INTEGER,  
    line_end INTEGER,  
    confidence_score REAL,  
    extracted_by TEXT  
);
```

Validation Result

```
CREATE TABLE validation_result (  
    validation_id TEXT PRIMARY KEY,  
    validation_type TEXT,  
    entity_type TEXT,  
    entity_id TEXT,  
    status TEXT,  
    message TEXT,  
    created_at TIMESTAMP  
);
```

Modernization Recommendation

```
CREATE TABLE modernization_recommendation (  
    recommendation_id TEXT PRIMARY KEY,  
    target_entity_type TEXT,  
    target_entity_id TEXT,  
    recommendation_type TEXT,  
    priority TEXT,  
    confidence_score REAL,  
    recommendation_text TEXT,  
    created_at TIMESTAMP  
);
```

Graph Materialization Views

MIP should expose graph-ready views.

Program Call View

```
CREATE VIEW vw_program_calls AS  
  
SELECT  
    source_id AS caller,  
    target_id AS callee  
  
FROM relationship  
  
WHERE relationship_type='CALLS';
```

Program Table View

```
CREATE VIEW vw_program_tables AS

SELECT *

FROM relationship

WHERE relationship_type IN
(
'READS_TABLE',
'WRITES_TABLE'
);
```

Job Program View

```
CREATE VIEW vw_job_programs AS

SELECT *

FROM relationship

WHERE relationship_type='EXECUTES';
```

Graph Export Strategy

SQLite

↓

Metadata

↓

Relationship Table

↓

NetworkX Graph Builder

↓

Knowledge Graph

Query Examples

Programs Calling TS2AUTH

```
SELECT *  
  
FROM relationship  
  
WHERE relationship_type='CALLS'  
  
AND target_id='TS2AUTH';
```

Programs Updating CARD_MASTER

```
SELECT *  
  
FROM relationship  
  
WHERE relationship_type='WRITES_TABLE'  
  
AND target_id='CARD_MASTER';
```

Jobs Executing TS2AUTH

```
SELECT *  
  
FROM relationship  
  
WHERE relationship_type='EXECUTES'  
  
AND target_id='TS2AUTH';
```

Enterprise Extensions (Future)

Version 2 adds:

```
api
screen
mq_queue
ims_transaction
business_rule
data_element
service_candidate
modernization_wave
risk_item
lineage_path
```

Scaling Strategy

Phase 0-3

SQLite

Suitable for:

- Development
 - POC
 - Hackathons
-

Phase 4+

PostgreSQL

Suitable for:

- Teams
 - Multi-user
 - Enterprise Deployment
-

Success Criteria

The schema must support:

✓ Metadata Storage

✓ Evidence Storage

✓ Validation Storage

✓ Relationship Storage

✓ Graph Construction

✓ Reasoning Engine

✓ Modernization Intelligence

without requiring schema redesign.

MIP Principle

Store facts.

Build relationships.

Generate intelligence.

End of Document.